LOCAL-FIRST DEVELOPER WORKSPACE · ENGINEERING REPORT

H

PROJECT REPORT · NUMBER 1

Helpyard
Local PC Control Dashboard

Project overview, system architecture, interface guide, implemented capabilities, verification evidence, risk assessment, and a staged path to a safe release.

Prepared: 17 October 2026
Reviewed baseline: local HTML dashboard + Windows PowerShell bridge
Report status: verified prototype assessment; public release not yet approved

Prepared from the project files and local checks available in this workspace. Capabilities are labelled as implemented, partially implemented, or planned; untested environments are not presented as verified.

Executive summary

Helpyard is a local workstation dashboard built as a single HTML page paired with a Windows PowerShell HTTP bridge. Its purpose is to bring system monitoring, project discovery, process/service controls, a command runner, local Ollama chat, and configuration into one browser interface.

1
Single-page frontend, no frontend build step

12
Navigation sections currently shown

20+
Local HTTP route patterns in the bridge

What is working in the verified environment: bridge authentication and system data routes; dashboard navigation and operational panels; project, process, and service lists; a read-only PowerShell command; Ollama model discovery; and an end-to-end local chat response. The previously discovered parser and Ollama response-shape defects were fixed and rechecked.

Release decision: suitable for controlled local evaluation, not public internet exposure. The bridge can execute commands and control Windows. Firefox was unavailable in the environment; full browser coverage, complete threat-model hardening, and a mock-backed public demo remain outstanding.

At a glance

Dimension	Current assessment	Evidence / interpretation
Architecture	Implemented	One HTML/CSS/JS UI and a localhost-bound PowerShell bridge; Ollama is optional.
Core local use	Verified in one environment	Windows 11, Windows PowerShell 5.1 and integrated Chromium-based VS Code browser.
AI	Local chat verified	Ollama model list and non-streaming chat response tested end to end.
Project intelligence	Basic discovery	Lists candidate folders and immediate Git repositories; not a repository analyzer/indexer.
Security posture	Local boundary present; hardening needed	Loopback binding, token and origin checks exist; arbitrary PowerShell execution is high impact.
Public readiness	Not ready	Do not publish a build connected to the workstation bridge.

Contents

1. Project purpose, scope, and user value
2. Architecture and request/data flow
3. Graphical dashboard walkthrough
4. Feature inventory and implementation status
5. Bridge/API reference and operational behaviour
6. Security, privacy, and trust boundaries
7. Verification performed and evidence
8. Known limitations and prioritized recommendations

9. Upgrade roadmap and acceptance gates
10. Installation, startup, and daily-use guide
11. Appendix: file inventory and glossary

1. Project purpose, scope, and user value

Purpose

Helpyard consolidates common workstation and developer operations in a navigable browser dashboard. The interface is intentionally a regular HTML document; system-level operations are delegated to a separate local PowerShell process because a browser page cannot safely or directly control Windows.

Intended audience and use cases

- A Windows developer who wants a quick view of CPU, memory, uptime, disks, processes, and services.
- A user who wants to browse a lightweight list of likely project folders and Git repositories.
- A local-AI user who wants to select an installed Ollama model and chat without sending that chat to a cloud provider.
- An operator who needs a browser UI for a PowerShell session, a small set of Windows actions, and service controls.

Design principles visible in the implementation

Local-first

Bridge listener binds to `127.0.0.1`; system APIs are served by the same machine. Ollama defaults to loopback.

Single-file UI

HTML, styles, and frontend JavaScript are together in `pc-control-dashboard.html`; no frontend package manager is required.

Visible controls

Sidebar navigation, connection badge, admin status, confirmation prompts, terminal output, and toast feedback expose operational state.

Incremental product direction

The upgrade plan prioritizes testability and safety before larger project indexing, editing, 3D visuals, or public hosting.

> **Scope boundary:** "Helpyard" is currently a workstation dashboard and a foundation for a developer workspace. Several navigation labels imply future integrations, but not all of them are connected to live backends yet. This report differentiates real behavior from display scaffolding.

2. Architecture and request/data flow

The design has three primary runtime components. The browser owns presentation and user interaction; the PowerShell bridge owns privileged/local operations; optional Ollama handles local inference. An optional cloud chat path calls provider APIs directly from the browser.

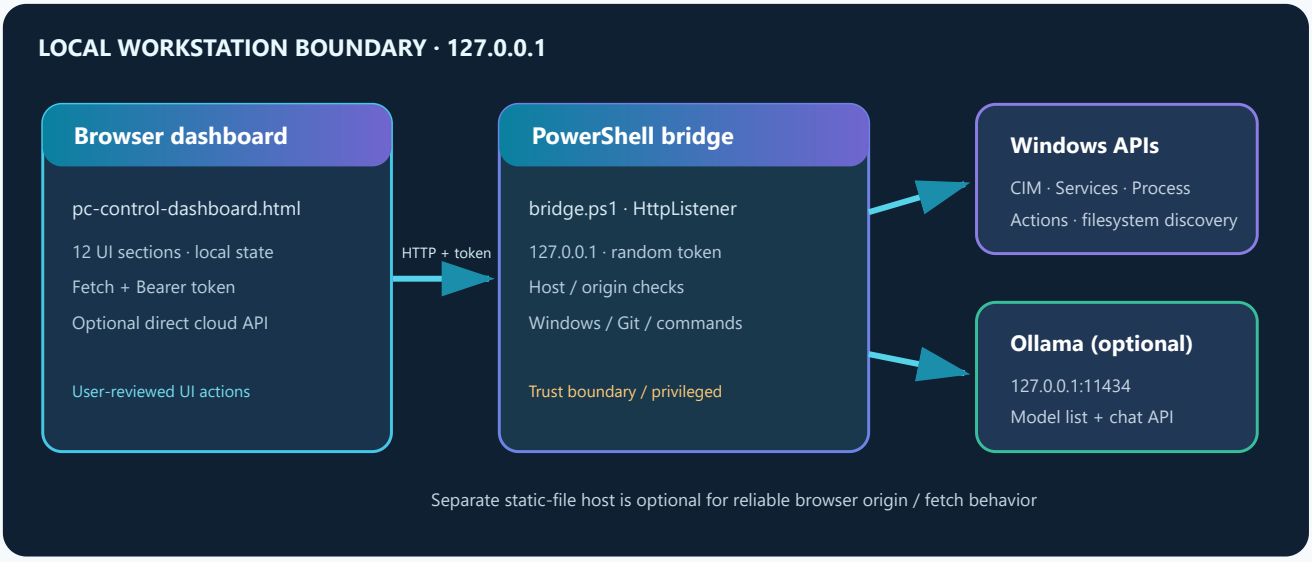


Figure 1. Logical architecture. The browser is not a security boundary for commands; authorization and permissions must be enforced by the bridge.

Typical request lifecycle

1 · User selects panel or command › **2 · Browser** sends HTTP + bearer token › **3 · Bridge** checks local host/origin + token › **4 · Windows / Ollama** executes or returns data › **5 · Dashboard** renders result or error

Runtime responsibilities

Component	Owns	Does not provide
Dashboard HTML	Layout, navigation, rendering, browser state, direct cloud chat requests, HTTP client.	Unrestricted filesystem or OS capability by itself; durable secure credential storage.
PowerShell bridge	Local APIs, Windows host metrics, action/service/process operations, arbitrary PowerShell route, Ollama proxy, Claude Desktop config access.	Remote-safe multi-user authorization, sandboxing for arbitrary commands, production audit management.
Ollama	Installed local model listing and inference on the local machine.	Guaranteed model availability, latency, or quality; model setup is external.
Static file server	Serves the HTML document over loopback when the user chooses.	API backend or authorization; it should remain bound to loopback for local use.

3. Graphical dashboard walkthrough

The live interface uses a dark navy palette, cyan/violet accent gradients, bordered cards, rounded status badges, a sticky navigation rail on wide screens, and a stacked layout at smaller widths. The following is a report illustration of the page structure and expected information hierarchy; it is a designed diagram, not a captured screenshot.



Figure 2. Illustrative overview screen. Actual live values are populated by bridge APIs; the drawing is explanatory and is not evidence of a particular machine reading.

Navigation map

Group	Screen	Primary user task
System	Overview	Review CPU/RAM, uptime, process count, disks, local model availability, tooling labels.
System	Projects	Refresh the bridge-discovered folder and Git repository cards.
System	Workbench	Edit and save assistant system instructions; see ready action cards.
System	Processes	Filter the top process list and request process termination with confirmation.
System	Services	Filter services and request start/restart/stop operations.
System	Terminal	Submit a PowerShell command and read its captured output.
AI	Local AI	Choose a model, refresh installed models, send local chat messages.
AI	Cloud AI	Select Anthropic/OpenAI/Gemini, set model and API key, chat directly with provider.
AI	MCP	View descriptive integration cards; live MCP server management is not wired to these cards.
AI	Skills	Run predefined PowerShell snippets through the same command endpoint.
Security	Security	See static posture/audit labels; this screen is not a live audit engine.
Admin	Settings	Set bridge URL/token; clear saved browser data.

Responsive and visual behaviour

- On wide screens the dashboard uses a 250-pixel sidebar and a flexible main area.
- At viewport widths at or below 900 pixels the sidebar moves above the page content.
- Cards use responsive auto-fit grids; tables are wrapped for horizontal overflow.

- The theme control changes the document background and browser color scheme. Some component styles remain based on the dark palette, so light-mode consistency should be checked in a dedicated visual pass.

4. Feature inventory and implementation status

Status definitions: **Live** means backed by a real local service route and exercised in the tested setup; **Partial** means implemented with important limitations or not fully tested; **UI only** means a label/card is present without the implied complete integration.

Capability	Status	What it currently does	Important boundary
System overview	Live	Reads OS/CPU/RAM/uptime/disks/process count, renders rings and CPU/RAM history.	Polls on interval; errors in the stats loader are silently caught in current frontend.
Project discovery	Partial	Scans a fixed set of roots and immediate child Git repositories.	Not a user-granted workspace picker, deep crawler, dependency graph, or source index. Broad discovery roots should be reviewed for privacy.
Workbench	Partial	Stores a context label and system prompt in browser state; renders predefined action cards.	Does not generate/track an implementation plan or gate every arbitrary command by the prompt.
Process manager	Live	Fetches 60 processes sorted by working set; filters list; asks before stop request.	Stop endpoint accepts a PID. Bridge-side target policy is minimal; operation affects the whole user machine.
Service manager	Live	Lists Windows services and allows start/restart/stop from UI.	Service mutation is consequential and permissions vary. Server route does not currently enforce per-service approval policy.
Terminal	High-impact live capability	Runs a supplied PowerShell script block and returns formatted output.	Authenticated user can submit arbitrary commands; this is not a sandbox and no UI confirmation provides containment.
Local AI	Live	Lists installed Ollama models and sends conversation turns through the bridge.	Current implementation is request/response, not streaming; requires local Ollama/model.
Cloud AI	Partial	Browser directly calls Anthropic, OpenAI, or Gemini APIs.	Key is saved in localStorage by current state handler. Browser origin/CORS/provider policy may limit use. Use only if intentional; avoid shared browser profiles.
MCP	UI only / partial backend	Dashboard displays sample service descriptions; bridge has MCP process start/status/stop routes.	No visible configuration form wires the cards to the bridge lifecycle API in this UI.
Skills	Live predefined commands	Three hard-coded snippets call the terminal route and show output.	These execute commands, not isolated skills; output displays in the terminal panel.
Security / audit	Static display	Shows explanatory security and audit status labels.	Labels are hard-coded UI content; there is no persistent action audit log or scan implementation behind them.
Claude Desktop config	Bridge API only	Bridge reads/writes the per-user Claude Desktop JSON config and backs up before write.	No corresponding configuration editor is exposed in current dashboard navigation.

> **Interpret "security posture" carefully.** The dashboard's static checklist is descriptive, not the result of an automated audit. It must not be used to claim that the workstation or application passed a security scan.

5. Bridge/API reference

Connection and authentication

- Default bridge port: `8787`. If busy, the script probes a short configured list of alternatives.
- Listener prefix: `http://127.0.0.1:<port>/`.
- A random 16-byte token is generated at startup when no token parameter is supplied; requests use `Authorization: Bearer <token>`.
- Host, remote IP, and non-null origin are checked for local request patterns; protected API routes call the shared authorization function.
- Ollama base URL defaults to `http://127.0.0.1:11434` and can be overridden with a script parameter.

Route inventory

Method + route	Purpose / data	Impact
`GET /api/ping`	Connection status, admin flag, computer and user name.	Local identity metadata
`GET /api/health`	OS caption/build and uptime.	System metadata
`GET /api/actions`	Action registry with permission, risk, approval and audit labels.	Read-only metadata
`GET /api/projects`	Candidate roots and nearby Git repositories, paths/branch/status.	Filesystem metadata
`GET /api/stats`	CPU, RAM, disk, uptime, host and process count.	System metadata
`GET /api/processes`	Top 60 processes by memory.	Process metadata
`POST /api/process/stop`	Force-stop a PID supplied in request JSON.	**Destructive**
`GET /api/services`	Enumerate Windows service name, display name, status and startup type.	System metadata
`POST /api/service`	Start, stop or restart a named Windows service.	**System mutation**
`POST /api/action`	Run a named allowlisted action; checks registration, approval and admin flag.	Varies; some require explicit approved field
`POST /api/run`	Compile and execute supplied PowerShell script block; returns text.	**Arbitrary code execution**
`GET /api/ollama/tags`	Proxy installed Ollama model list.	Local inference metadata
`POST /api/ollama/chat`	Proxy chat request; forces non-streaming mode for JSON response handling.	Local inference
`GET /api/mcp/status`	Show process IDs/running state for bridge-managed processes.	Process metadata
`POST /api/mcp/start`	Start a supplied executable with args and optional environment values.	**Process launch**
`POST /api/mcp/stop`	Stop a bridge-tracked process.	**Process termination**

Method + route	Purpose / data	Impact
`GET /api/claude-config`	Read Claude Desktop config JSON from the current user profile.	Potentially sensitive config
`POST /api/claude-config`	Validate JSON, make timestamped backup, then write config.	User config write

Action registry

Category	Examples	Current guard indicated in registry/route
Low-risk utilities	Lock, cancel shutdown, flush DNS, Task Manager, File Explorer, Settings, Device Manager, Windows Update.	Most are marked standard permission; no approval required.
Power changes	Sleep, restart, shutdown; performance profiles.	Sleep/restart/shutdown and high-performance profile are flagged approval-required; some require admin.
Cleanup	Empty recycle bin; clean user temp directory.	Approval-required in registry; destructive file operations.

> **Important implementation gap:** the bridge exposes more powerful routes than the UI action registry implies. In particular, `/api/run`, service changes, PID termination, and MCP process launch are bearer-token protected but are not restricted by an allowlisted command policy. A token holder must be treated as a highly trusted local operator.

6. Security, privacy, and trust boundaries

Existing controls

- Loopback-only listener prefix limits direct access to the host machine and avoids an intentional LAN bind.
- Random startup token provides an access check for protected endpoints.
- Host, client address, and origin checks attempt to limit requests to local browser contexts.
- Allowlisted system actions include risk/permission/approval metadata, and the server checks approval/admin status for those actions.
- Claude config writes create a timestamped backup before replacing the file.
- The README warns not to publish the bridge and recommends loopback serving for the dashboard.

Risk heat map

Risk	Likelihood / impact	Why it matters	Recommended treatment
Arbitrary command execution by token holder	Critical impact	`/api/run` executes caller-supplied PowerShell with the bridge user's permissions.	Disable by default or replace with named allowlisted tasks, restricted arguments, timeouts, output caps, and explicit server-side policy.
Browser token / cloud key in localStorage	High	Current state object saves bridge token and cloud key in the browser origin's local storage.	Do not reuse browser profiles; minimize persistence; separate cloud keys; document origin/XSS exposure; consider OS-backed secret storage.
Wide Windows management routes	High	Service/process/MCP routes accept mutation requests without a central least-privilege action policy.	Validate request schema and targets; add explicit approval on server and auditable allowlists.

Risk	Likelihood / impact	Why it matters	Recommended treatment
Misleading static security indicators	Medium	Security and audit UI rows are hard-coded, not derived from event data.	Label as guidance until actual checks and durable audit events exist.
Silent dashboard failures	Medium	Some fetch/load code catches and suppresses exceptions; users may see stale/placeholder metrics.	Expose per-panel last-updated, error message, and retry states; never present stale values as fresh.
Local file discovery scope	Medium	Project discovery roots include profile/Desktop/program directories and process working directory; paths reveal user data layout.	Use explicit approved workspaces; least-data default; redact paths in exports.
Public bridge exposure	Unacceptable	Machine-control API is not a hosted multi-user service; network exposure could transfer control.	Never publish the real bridge. Public demo must use synthetic backend and no machine capability.

Privacy/data handling

- Bridge responses include computer name, user name, CPU/memory/disk usage, process names, service inventory, candidate folder paths, and optionally project Git state.
- Local AI prompts go to the configured local Ollama server by default; the bridge forwards chat content and model name.
- Cloud chat sends prompt contents and an API key directly from the browser to the selected third-party provider. The cloud key is persisted in the current browser state implementation.
- No code in the reviewed project establishes a telemetry or analytics system. The absence of observed telemetry is not a substitute for a documented release privacy review.

> **Safe deployment model:** keep the workstation UI/bridge private and local. If sharing a demo publicly, run a separate version with mock data and no routes to local processes, files, services, shell, models, or credentials.

7. Verification performed and evidence

This section records the checks actually performed in the available Windows environment. It does not claim comprehensive automated testing or certification across every browser or OS.

Check	Observed result	Evidence boundary
Windows PowerShell syntax	Pass Windows PowerShell 5.1 AST parser reported no parse errors after the authorization fix and after the Ollama route update.	One PowerShell runtime; PowerShell 7 compatibility not exercised.
Frontend inline JavaScript parse	Pass Extracted inline script compiled with Node.js `new Function`.	Syntax check does not prove all runtime paths.
Bridge authentication	Pass Valid token ping returned HTTP 200; absent and incorrect token returned HTTP 401.	Local loopback smoke test only.
Origin check	Pass A request with an invalid external Origin was rejected with HTTP 403.	Does not replace a full browser-origin / CSRF assessment.
Read APIs	Pass Stats, projects, processes, and services returned HTTP 200 in local smoke tests.	Host-specific results; no load or stress tests.

Check	Observed result	Evidence boundary
Integrated browser navigation	**Pass** All 12 page headings were traversed in the VS Code integrated browser.	Browser user-agent reported Chromium 150 / Electron 43 on Windows 11; not a matrix test of installed Edge/Chrome/Firefox products.
Operational UI	**Pass** Overview metrics, discovered projects, 60 process rows, service rows, and local model list populated.	Representative workstation only.
Terminal	**Pass** Read-only command `Get-Date -Format yyyy-MM-dd` returned output and a completed state.	Does not make arbitrary command execution safe.
Local AI chat	**Pass after fix** Local selected model returned the exact test response `READY` through dashboard → bridge → Ollama.	One installed model and one short prompt; no throughput/quality tests.
Firefox	**Not tested** Firefox executable was not present in the test environment.	No claim of Firefox support validation.

Defects found and corrected during the project work

1. **PowerShell parser failure:** malformed token comparison and duplicate authorization helper prevented the bridge script from parsing. Replaced it with one valid check and updated the request handler to use that function.
2. **Local AI empty responses:** dashboard read a single JSON object, but the bridge forwarded Ollama's default stream mode. The bridge now requests `stream = false` and returns the complete response; the UI chat was then verified.
3. **Startup instructions:** corrected README and settings guidance so normal startup uses a regular PowerShell session and explains the user-scoped execution-policy bypass.

> **Not performed:** full unit/automation test suite, Windows service mutation tests, reboot/shutdown tests, external network exposure, adversarial security review, Firefox UI run, and public staging deployment.

8. Known limitations and recommendations

Immediate P0 — required before broader local rollout

1. **Contain command execution.** Replace general-purpose `/api/run` with named safe operations, configurable allowlists, and argument schemas. If unrestricted shell access is intentional, make the trusted-user risk explicit and disable it by default.
2. **Unify authorization and policy.** Every mutating route (actions, service, process, MCP, config write, command execution) should pass a central policy function with role, target, approval, and audit metadata.
3. **Bound request and execution resource use.** Set request body size caps, command duration/timeouts, output size limits, and cancellation behavior. Registry timeout/output labels are metadata today and are not consistently enforced by the routes.
4. **Make errors visible.** Remove silent catches in stats/provider flows; add last-success timestamp, error message, retry, and stale-data indication to each panel.
5. **Protect browser state.** Handle invalid JSON/corrupt localStorage and browser storage denial. Avoid storing cloud API keys and long-lived bridge credentials unnecessarily; provide an explicit clear/logout flow.
6. **Review discovery and configuration endpoints.** Limit project discovery to user-approved roots; restrict Claude config read/write to an intentional UI, with backups and clear confirmation.

P1 — release quality

- Add regression tests for parser cleanliness, auth, hostile origin, malformed JSON, unknown route, invalid action, approval/admin denial, request limits, and shutdown cleanup.
- Exercise the supported browser versions on Windows at desktop and narrow viewport sizes; record actual version, result, and screenshot/test evidence.
- Implement keyboard-visible focus, accessible labels, reduced-motion support, error announcements, responsive table handling, and a deliberate light-theme palette.
- Distinguish actual tooling discovery from static “Available locally” claims.
- Add real audit events with timestamps, action, target, caller context, decision and result; avoid recording secrets or full command content where unnecessary.

P2/P3 — product expansion

- Explicit project folder selection, safe file tree, manifest/language detection, secret-aware excludes, searchable local index, and source-referenced answers.
- Patch-based editor workflow with diff review, per-file approval, backups, apply/reject/revert and verified test outcomes.
- Ollama health state, streaming with cancellation, history persistence controls, model settings, and timeouts.
- Wire MCP server lifecycle UI to bridge routes only after executable allowlisting and isolation are specified.
- Create a separate public demo mode with synthetic data. Do not tunnel or forward the live local bridge.

9. Upgrade roadmap and acceptance gates

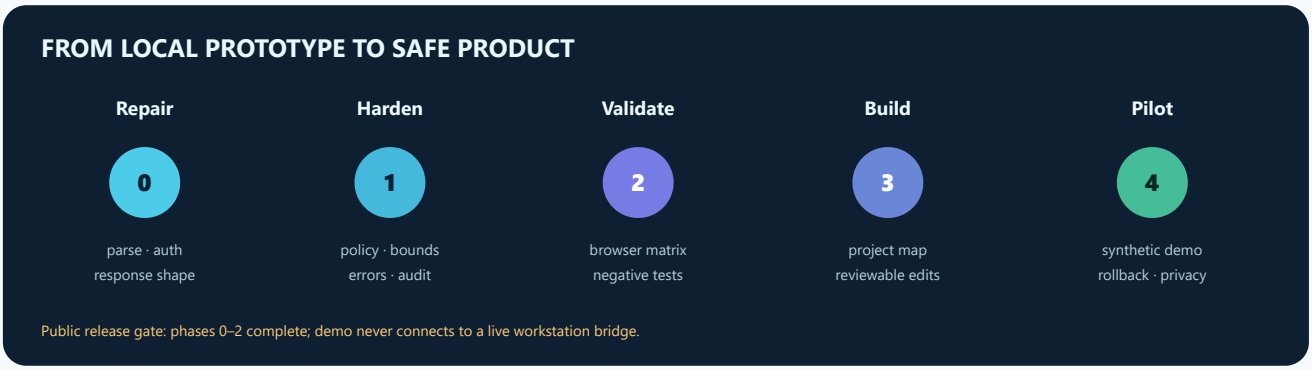


Figure 3. Sequencing avoids building feature breadth on top of an unsafe or untestable execution foundation.

Phase	Deliverables	Acceptance gate	Priority
0 · Stabilize	Parser/auth/chat fixes, startup steps, repeatable smoke test.	Parse clean; valid/invalid auth covered; live local chat works; known error states documented.	P0 · in progress / partly verified
1 · Secure	Central policy, restricted commands, input/output/time bounds, error reporting, audit log, storage cleanup.	Mutation routes have server-side policy; rejected requests do not alter machine state; audit evidence exists.	P0
2 · Qualify	Browser/device matrix, accessibility and responsive pass, install/stop docs, regression automation.	Supported browser list and recorded test evidence; no blocker defects in representative offline/connected cases.	P1
3 · Workspace	Explicit allowed folders, safe project explorer, metadata index, model-assisted cited context.	Traversal and secret-exclusion tests pass; user can clear context/history.	P2
4 · Edit safely	Diff-based patches, approvals, backup/rollback, controlled test execution.	Rejected patch never writes; accepted patch is recoverable; test exit status is accurate.	P2

Phase	Deliverables	Acceptance gate	Priority
5 · Public pilot	Isolated mock deployment, privacy statement, support/feedback, rollback, release notes.	No bridge credentials or workstation endpoint in demo; synthetic-data tests and rollback pass.	P3

> **Go/no-go:** no public launch of the machine-control bridge. Approve a public-facing pilot only after P0/P1 gates and only against a separate mock or sandbox service.

10. Installation, startup, and daily-use guide

Prerequisites

- Windows with Windows PowerShell; regular user permissions for normal read/status use.
- A current browser. The UI was exercised in the VS Code integrated Chromium browser; other browser products should be tested directly before claiming support.
- Optional: Python for the simple static HTTP server, or another local static-file host. The bridge itself does not require Node.js.
- Optional: Ollama reachable at `http://127.0.0.1:11434` with at least one pulled model.

Start the bridge

Open a regular PowerShell terminal in the project folder and start the bridge using a process-scoped execution-policy bypass:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\bridge.ps1
```

The bridge prints the listening URL and a generated bearer token. Keep the terminal open while the bridge is in use. Stop it with `Ctrl+C`.

Serve and open the dashboard

In another terminal, serve this directory on loopback:

```
python -m http.server 8000 --bind 127.0.0.1
```

Open `http://127.0.0.1:8000/pc-control-dashboard.html` in the browser. Serving over localhost provides predictable browser origin/fetch behavior.

Connect and operate

1. Open Settings from the sidebar.
2. Enter the bridge URL printed in PowerShell (default port 8787 unless the selected port was changed).
3. Paste the generated token and select Save & connect. The sidebar should show Connected and the host name.
4. Use Overview first; navigate to Projects, Processes, Services, or Local AI only as needed. Refresh buttons reload related data.
5. Use Terminal and action buttons only for commands/actions understood and intentionally authorized. A browser confirmation does not sandbox or neutralize the command.
6. Clear saved browser data from Settings when finished on a shared profile. Stop the bridge and static server when done.

Common recovery

Symptom	Check	Response
Sidebar stays Offline	Bridge process, URL/selected port, token, local server origin.	Copy the latest bridge URL/token from startup output and reconnect in Settings.
Ollama Offline / no models	Ollama service and model list endpoint.	Start Ollama, pull a model, refresh Local AI; dashboard remains usable without AI.
Administrator-required action rejected	Bridge says the current process is not elevated.	Use the standard account for routine work. Elevate only if the specific action is required and understood.
Port conflict	Bridge startup output reports selected fallback port.	Use the printed URL, not the default, in Settings.
Browser cannot fetch bridge	Verify both services are on localhost and inspect browser console/network.	Serve the HTML on loopback; avoid remote origins and do not broaden bridge binding.

11. Project files and glossary

Reviewed and generated files

File	Role	Report note
`pc-control-dashboard.html`	Application interface, styling and browser logic.	Single-file dashboard; main user entry point.
`bridge.ps1`	Windows localhost HTTP service and route handlers.	Privileged trust boundary; must not be public-hosted.
`README.md`	Startup, safety and validation guidance.	Updated to reflect regular-user startup and loopback serving.
`update plan no1.txt`	Original product vision and broad implementation ideas.	Describes future project intelligence, editor and 3D workspace goals; those goals are not all implemented.
`Plannumber1.Pdf`	Numbered upgrade and launch plan.	Prior companion deliverable; execution gates and phase plan.
`Plannumber1.html`	Editable HTML source for the prior plan PDF.	Print-ready source for revision.
`Project Report Number1.html`	Editable source for this report.	Contains vectors/diagrams and presentation styling.
`Project Report Number1.Pdf`	Requested presentation/report artifact.	Generated from this HTML source for sharing/printing.

Glossary

Term	Meaning in this project
Bridge	The PowerShell process that exposes a local HTTP API to the browser.
Loopback	Local host-only network interface, typically 127.0.0.1; not intended for remote users.
Bearer token	Secret string sent in the Authorization header to authenticate bridge requests.
Ollama	Optional local service for listing and running installed language models.
MCP	Model Context Protocol; a potential integration area, only partly represented in the current interface.

Term	Meaning in this project
LocalStorage	Browser-origin key/value storage; convenient but not equivalent to a secure credential vault.
Mock backend	Safe substitute that returns synthetic data without reaching a user's real workstation.

Final assessment

Helpyard has a compact, understandable architecture and demonstrates useful local workstation workflows. The live system/bridge and local AI path were verified in one Windows Chromium-based environment. Its next success criterion is not more screens; it is consistently enforcing least privilege, surfacing errors accurately, and capturing repeatable cross-browser evidence. Treat the current build as a local prototype until those controls and release gates are completed.